AMRITA VISHWA VIDYAPEETHAM
DEEMED TO BE UNIVERSITY

DATASTRUCTURES AND ALGORITHMS

23CSE203

LAB MANUAL

SUBMITTED BY	
NAME	K. Naga Venkata Sruthi
ROLL NO	AV.SC.U4CSE25123
YEAR/SEM/SECTION	2ndYEAR/ SEM-3/CSE-B
CREDITS	05

SUBMITTED TO	DR. RAJ KUMAR BATCHU
DESIGNATION	ASST.PROFFESSOR(SL Grade)
DEPARTMENT	SCHOOL OF COMPUTING

INDEX

S.NO	TITLE	PAGE NO	REMARKS
1	WEEK 1: A. A rocket launch system displays a countdown before launch. Instead of using a loop implement the countdown using recursion. Write a recursion python program to display a countdown from n to 1 followed by “launch”. B. The banks calculate the growth of an investment using compound interest; the formula requires calculating powers. Which can be implemented recursively. Write a recursive program to calculate p^n(power), where p=principal growth factor, n= number of years. C. An HIR department stores employee IDs in a list. Develop a recursive program to search for a given employee ID. Write a recursive function to search for an employee ID in a list. D. In a logistics company the number of possible ways to arrange n different parcels are calculated using the factorial formula(n!). develop a recursive program to calculate the factorial of a given number. E. Write a recursive python program to print the first n terms of the Fibonacci series.		

INDEX

S.NO	TITLE	PAGE NO	REMARKS
2	WEEK 2: A. Write a program to perform linear search on a given input. B. (i) Write a python program to implement binary search with sorted array as an input. (ii) Write a python program to implement binary search without sorted array as an input		
3.	WEEK-3: A.Write a python program to implement bubble sort, where the input elements are taken from the user. B. Write a python program to implement insertion sort, where the input elements are taken from the user. C. Write a python program to implement selection sort, where the input elements are taken from the user.		
4	WEEK 4 : A.Write a python program to implement merge sort where the input elements are taken from the user B. Write a python program to implement quick sort where the input elements are taken from the user		

INDEX

S.NO	TITLE	PAGE NO	REMARKS
5.	WEEK 5 : a) Write a Python program to implement a singly linked list with the below operations: 1. Create linked list 2. Insert at beginning 3. Insert at end 4. Insert at a specificindex 5. Delete by value 6. Delete first node 7. Delete last node 8. Count number of nodes 9. Display / Traverse 10. Exit b) Execute above programs for doubly linked list c) Execute above programs for Circular linked list		
6.	WEEK-6 Write a Python program to implement stack operations such as **Push, Pop, Peek, Display** using an **array and a linked list**.		
7.	WEEK-7 a) Write a Python program to implement a **queue** and its operations such as **Enqueue, Dequeue, Peek, Display** with a **menu-driven approach**. Implement using **array and linked-list data structures**.		

S.NO	TITLE	PAGE NO	REMARKS
	b) Circular Queue- its operations such as **Enqueue, Dequeue, Peek, Display** with a **menu-driven approach**. Implement using **array and linked-list data structures**.		
8.	WEEK-8 Python program to implement a binary tree and its operations using an array and a linked list.		
9	WEEK-9 Python program to implement a binary tree with its operations and traversals.		

Week-1

A.AIM: Arocket launch systemdisplays a countdown before launch. Instead of using a loop implement the countdown using recursion. Write a recursion python program to display a countdown from n to 1 followed by “launch”.

PROGRAM:

```
def launch(n):

    for i in range (n, -1,-1):

        if i==0:

            print("launch")

        else:

            print(i)

n=int (input ("Enter no. of seconds to launch:"))

launch(n)
```

OUTPUT:

```
===== RESTART: C:\Users\Sruti\AppData\Local\Programs\Python\Python314\1a.py ===
Enter no. of seconds to launch: 5
5
4
3
2
1
launch
```

EXPLANATION:

This program displays a countdown from n to 1 and finally prints 'launch'. Although the aim mentions recursion, the given code uses a loop.

ERROR TABLE:

S.no.	Errors	Rectification
1.	Indentation Error	Use 4 spaces for each level of indentation.
2.	Type Error	Convert the input to an integer using n = int (input ("Enter no. of seconds to launch: ")).

B.AIM: The banks calculate the growth of an investment using compound interest; the formula requires calculating powers. Which can be implemented recursively. Write a recursive program to calculate p^n(power), where p=principal growth factor, n= number of years.

PROGRAM:

```
p = float (input ("Enter principal growth factor (p): "))
n = int (input ("Enter number of years (n): "))
def power (p, n):
    if n == 0:
        return 1
    else:
        return p * power (p, n - 1)
result = power (p, n)
print ("Power: ", result)
```

OUTPUT:

```
3.14.0 (tags/v3.14.0:ebf955d, Oct  7 2025, 10:15:03) [MSC v.194
help" below or click "Help" above for more information.

ESTART: C:\Users\Sruti\AppData\Local\Programs\Python\Python314\
rincipal growth factor (p): 36
umber of years (n): 23
 6.2367382520429335e+35
```

EXPLANATION:

- The function of power (p, n) calculates p^n using recursion by multiplying p with the result of power (p, n-1).
- The recursion stops when n becomes 0, returning 1, which is the base case.
- The final result is displayed after all recursive calls are completed.

ERROR TABLE:

S.no.	Errors	Rectification
1.	Type Error	Convert the input to float or int using float (input()) or int (input ()).
2.	Incorrect output if the recursive call is written as power (p, n + 1) instead of power (p, n - 1).	Change the recursive call to power (p, n - 1) so that n decreases toward the base case.

C.AIM: An HIR department stores employee IDs in a list. Develop a recursive program to search for a given employee ID. Write a recursive function to search for an employee ID in a list.

PROGRAM:

```
def search (list, key, index):

    if index >= len(list):

        return False

    if list[index] == key:

        return True

    return search (lst, key, index + 1)

employees = [101, 102, 103, 104, 105]

key = int (input ("Enter Employee ID to search: "))

if search (employees, key, 0):

    print ("Employee ID Found")

else:

    print ("Employee ID Not Found")
```

OUTPUT:

```
"help" below or click "Help" above for more information.

RESTART: C:\Users\Sruti\AppData\Local\Programs\Python\Python314\1c.p
Employee ID to search: 1022
yee ID Not Found

RESTART: C:\Users\Sruti\AppData\Local\Programs\Python\Python314\1c.p
Employee ID to search: 105
yee ID Found
```

EXPLANATION:

- The recursive function search () checks each employee ID in the list one by one, starting from the given index.
- If the current employee ID matches the required ID (key), it returns True.
- If the ID does not match, the function calls itself recursively with the next index (index + 1).

- If the index reaches the end of the list, it returns False, indicating that the employee ID was not found.
- Finally, the if-else statement displays whether the employee ID is **Found** or **Not Found**.

ERROR TABLE:

S.no.	Errors	Rectification
1.	Name Error if the list name is misspelled (e.g., lst instead of list).	Use the correct variable name consistently throughout the program.
2.	Syntax Error due to a missing colon (:) after an if statement or function definition.	Add the missing colon (:) after if, else, or def.

D.AIM: In a logistics company the number of possible ways to arrange n different parcels are calculated using the factorial formula(n!). develop a recursive program to calculate the factorial of a given number.

PROGRAM:

```
def factorial(n):

    if n == 0 or n == 1:

        return 1

    else:

        return n * factorial (n - 1)

num = int (input ("Enter a number: "))

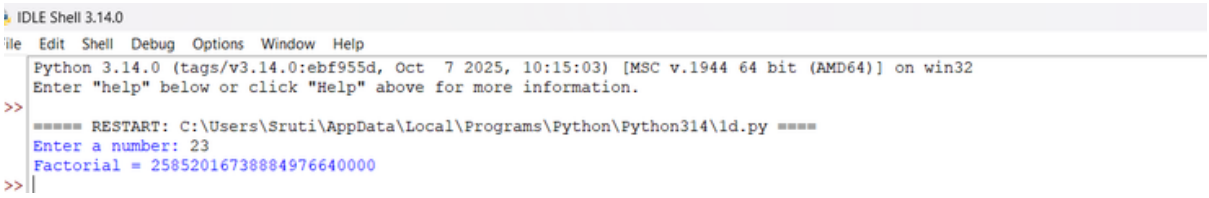
if num < 0:

    print ("Factorial does not exist for negative numbers.")

else:

    print ("Factorial =", factorial(num))
```

OUTPUT:



EXPLANATION:

- The function factorial(n) computes the factorial by multiplying the current number with the factorial of the previous number.
- The recursion ends when n becomes 0 or 1, where the factorial value is 1 .
- The program also checks for negative inputs since factorials are defined only for non-negative integers.

ERROR TABLE:

S.no	Errors	Rectification
1.	**Name Error** occurswhen a variable orfunctionisused without defining it first.	Define thevariableorfunction before using it in theprogram.
2.	**Value Error** occurs when an invalid value,suchasaletter, is entered whereanumberis expected.	Enter a valid numericalvalue or use proper inputvalidation.

E.AIM: Write a recursive python program to print the first n terms of the Fibonacci series.

PROGRAM:

```
n = int (input ("enter number of terms: "))

def fibonacci(num):

    if num==0:

        return 0

     elif num==1:

        return 1

    else:

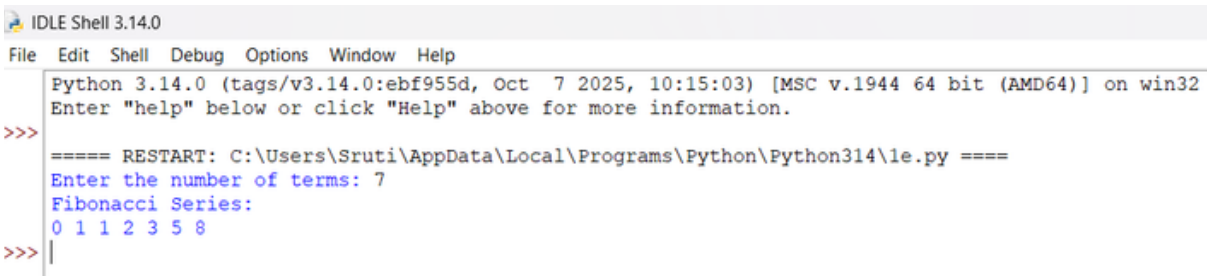
        return fibonacci(num-1) +fibonacci(num-2)

print ("Fibonacci series: ")

for i in range(n):

    print(fibonacci(i), end=" ")
```

OUTPUT:



EXPLANATION:

- The function Fibonacci(n) returns the nth Fibonacci number by recursively adding the previous two Fibonacci numbers.
- The base cases return 0 for the first term and 1 for the second term.
- A for loop calls the recursive function repeatedly to print the first n terms of the Fibonacci series.

ERROR TABLE:

S.no.	Errors	Rectification
1.	RecursionError if the base cases (n==0andn== 1) are not included.	Add both base conditions before the recursive call.
2.	ValueErrorifthe user enters text insteadofaninteger.	Enter a valid integer for the number of terms.

Week-2

A.AIM: Writeaprogramtoperformlinearsearch on a given input.

PROGRAM:

```
def linear_search(arr,key):

    for i in range(len(arr)):

        if arr[i]==key:

            return i

    return -1

 n = int (input ("Enter no. of elements:"))

arr= []

print ("Enter the elements:")

for i in range(n):

    arr.append(int(input ()))

key=int (input ("Enter element to search:"))

result=linear_search(arr,key)

if result! =-1:

    print ("Element found at index", result)

else:

    print ("Element not found.")
```

OUTPUT:

```
Python 3.14.0 (tags/v3.14.0:ebf955d, Oct  7 2025, 10:15:03) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>
===== RESTART: C:\Users\Sruti\AppData\Local\Programs\Python\Python314\2a.py ====
Enter no. of elements: 7
Enter the elements:
40
31
5
36
23
44
7
Enter element to search: 5
Element found at index 2
>>
```

EXPLANATION:

- The linear_search() function checks each element of the list one by one and compares it with the given key.
- If the key is found, its index is returned; otherwise, the function returns -1.
- The program takes the array and search element as input and displays whether the element is found.

ERROR TABLE:

S.no.	Errors	Rectification
1.	ValueErroroccurs if a non-integer value is enteredwhen int (input ()) is used.	Enter only integer values for the number of elements and array elements.
2.	TypeError mayoccur if len(arr) is used on a variable that is not a list.	Make sure arr is initialized as a list using arr = [] and elements are appended to it.

B(i). AIM: Write a python program to implement binary search with sorted array as an input.

PROGRAM:

```
def binary_search(arr, key):

    low = 0

    high = len(arr) - 1

    while low <= high:

        mid = (low + high) // 2

        if arr[mid] == key:

            return mid

        elif arr[mid] < key:

            low = mid + 1

        else:

            high = mid - 1

    return -1

n = int (input ("Enter number of elements: "))

arr = []

print ("Enter elements in sorted order:")

for i in range(n):

    arr.append(int(input ()))

key = int (input ("Enter the element to search: "))

result = binary_search(arr, key)

if result != -1:

    print("Element found at index", result)

else:
```

```
print("Element not found")
```

OUTPUT:

```
Python 3.14.0 (tags/v3.14.0:ebf955d, Oct  7 2025, 10:15:03) [MSC v.1944 64 bit (AMD64)]
Enter "help" below or click "Help" above for more information.
>
===== RESTART: C:\Users\Sruti\AppData\Local\Programs\Python\Python314\2b.py ====
Enter no. of elements: 4
Enter the elements:
7
5
24
48
Enter element to search: 48
Element found at index 3
>
===== RESTART: C:\Users\Sruti\AppData\Local\Programs\Python\Python314\2b.py ====
Enter no. of elements: 4
Enter the elements:
7
5
24
48
Enter element to search: 26
Element not found.
```

EXPLANATION:

- The program defines a binary_search() function that takes a sorted list and a search element as input.
- The variables low and high are initialized to represent the first and last positions of the list.
- The program calculates the middle position using mid = (low + high) // 2 and compares the middle element with the search element.
- If the search element is greater or smaller than the middle element, the program searches only the required half of the list.
- If the element is found, its index is returned; otherwise, -1 is returned and **"Element not found"** is displayed.

ERROR TABLE:

S.no.	Errors	Rectification
1.	**Indentation Error** if the statements inside the function or loop are not properly indented.	Maintain proper indentation inside the function and loop.
2.	**ValueError** if a non-numeric value is entered instead of an integer.	Enter only valid integer values.
3.	Incorrect result if the elements are not entered in sorted order.	Enter the elements in sorted order before performing binary search.
4.	**IndexError** if an invalid index is accessed while searching.	Make sure low and high remain within the list range.

B(ii). AIM: Write a python program to implement binary search without sorted(unsorted) array as an input

PROGRAM:

```
def binary_search(arr, key):

    arr.sort()

    low = 0

    high = len(arr) - 1

     while low <= high:

        mid = (low + high) // 2

         if arr[mid] == key:

            return mid

         elif arr[mid] < key:

            low = mid + 1

        else:

            high = mid – 1

    return -1

 n = int (input ("Enter number of elements: "))

arr = []

print ("Enter elements:")

for i in range(n):

    arr.append(int(input()))

key = int (input ("Enter the element to search: "))

result = binary_search(arr, key)

 if result != -1:

    print ("Element found at index", result)
```

```
else:
    print ("Element not found")
```

OUTPUT:

```
.4.0 (tags/v3.14.0:ebf955d, Oct  7 2025, 10:15:03) [MSC v.1
.p" below or click "Help" above for more information.

ART: C:\Users\Sruti\AppData\Local\Programs\Python\Python314
orted array: 22 33 66 43 99
ient to search: 66
:ay: [22, 33, 43, 66, 99]
ound at index: 3

ART: C:\Users\Sruti\AppData\Local\Programs\Python\Python314
orted array: 22 33 66 43 99
ient to search: 31
:ay: [22, 33, 43, 66, 99]
ot found
```

EXPLANATION:

1. The binary_search() function is defined to search for the given key in the list arr.
2. The arr.sort() function arranges the elements in ascending order so that binary search can be performed.
3. The variables low and high represent the first and last positions of the sorted list.
4. The program finds the middle element and compares it with the key. Based on the comparison, it searches either the left half or the right half.
5. If the element is found, its index is returned. If the element is not found, the function returns -1.

ERROR TABLE:

S.no.	Errors	Rectification
1.	**IndentationError** may occur if while, if, or other statements are not properly aligned.	Maintain proper indentation inside the function and conditional blocks.
2.	**SyntaxError** may occur if – is used instead of the normal minus operator - in high = mid – 1.	Use the normal hyphen: high = mid - 1.
3.	**ValueError** occurs if a non-numeric value is entered instead of an integer.	Enter only valid integer values.
4.	**NameError** occurs if variables such as arr, key, or result are misspelled.	Check the spelling and define variables before using them.

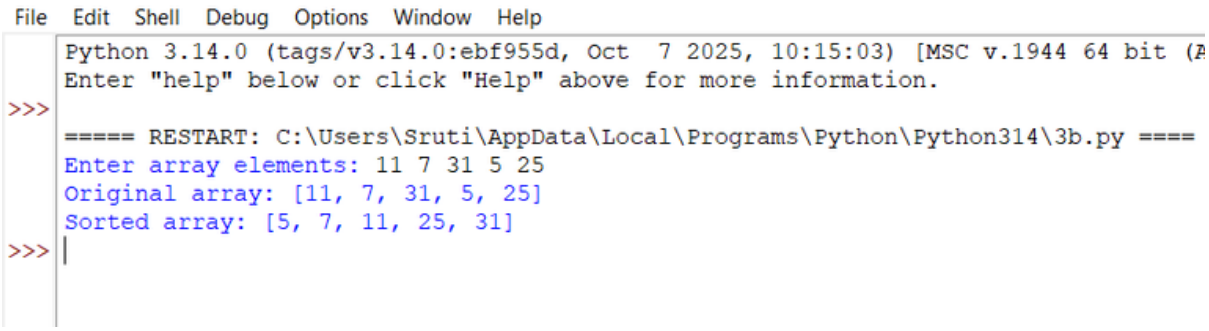
Week-3

A.AIM: Write a python program to implement bubble sort, where the input elements are taken from the user.

PROGRAM:

```
#bubble sort
def bubble_sort(arr):
    n = len(arr)
    for i in range (n-1):
        for j in range(n-i-1):
            if arr[j]>arr[j+1]:
                arr[j],arr[j+1] = arr[j+1],arr[j]
    return arr
#taking input
n = int(input("Enter number of elements: "))
arr=[]
print("Enter elements: ")
for i in range(n):
    arr.append(int(input()))
bubble_sort(arr)
print("sorted array: ")
for element in arr:
    print(element, end=" ")
```

OUTPUT:



EXPLANATION:

- The bubble_sort() function is defined to arrange the elements of the list in ascending order.
- The outer for loop controls the number of passes, while the inner for loop compares adjacent elements.
- The condition arr[j] > arr[j+1] checks whether the current element is greater than the next element.
- If the elements are in the wrong order, they are exchanged using arr[j], arr[j+1] = arr[j+1], arr[j].
- After all comparisons and swaps are completed, the sorted array is returned and displayed element by element.

ERROR TABLE:

S.no.	Errors	Rectification
1.	**ValueError**ifanon-integer value is entered asanarray element.	Enter only valid integer values and linely
2.	**NameError**ifarr, n, or another variable ismisspelled.	Check the variable names and use the correct spelling.
3.	**TypeError**mayoccur if the list contains incompatible data types during comparison.	Enter elements of the same suitable data type, such as integers.
4.	**IndexError** may occur if the program accesses an invalid array position.	Use the correct loop range range(n-i-1) to keep the index within the list.

B.AIM: Write a python program to implement insertion sort, where the input elements are taken from the user.

PROGRAM:

```
def insertion_sort(arr): n = len(arr) for i in range (1, n): key = arr[i] j = i - 1


  while j >= 0 and arr[j] > key:
      arr[j + 1] = arr[j]
      j = j - 1
   arr[j + 1] = key
return arr
#Taking input


n = int (input ("Enter number of elements: ")) arr = []

print ("Enter elements: ")

for i in range(n):

    arr.append(int(input()))

#Sorting     the     array

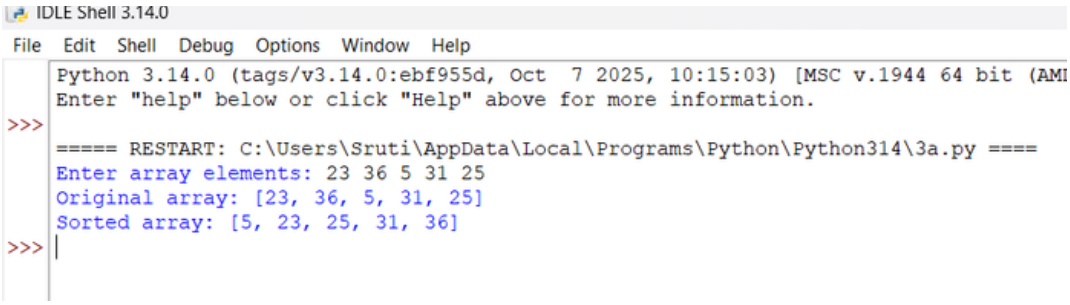
insertion_sort(arr) print

("Sorted   array:   ")   for

element in arr:

      print (element, end=" ")
```

OUTPUT:



EXPLANATION:

- The insertion_sort() function is defined to arrange the elements of the list in **ascending order**.
- The for loop starts from the second element because the first element is considered already sorted.
- The variable key stores the current element, and j is used to compare it with the previous elements.
- The while loop shifts elements that are greater than key one position to the right.
- The key is placed in its correct position, and after all elements are processed, thesorted array is displayed.

ERROR TABLE:

S.no.	Errors	Rectification
1.	**IndentationError** if the statements inside the function, for, or while loop are not properly indented.	Maintain proper indentation inside all blocks.
2.	**ValueError** if a non-integer value is entered as an array element.	Enter only valid integer values.
3.	**NameError** if variables such as arr, key, or j are misspelled.	Check the variable names and use the correct spelling.
4.	**IndexError** if an invalid list index is accessed while shifting elements.	Make sure j >= 0 is checked before accessing arr[j].

C.AIM: Write a python program to implement selection sort, where the input elements are taken from the user.

PROGRAM:

```
#SelectionSort

defselection_sort(arr): n = len(arr)

foriinrange(n - 1):
    min_index = i
    forjinrange(i + 1, n):
        ifarr[j] < arr[min_index]:
            min_index = j
    arr[i],arr[min_index] = arr[min_index], arr[i]
returnarr


#Takinginput

n = int(input("Enter number of elements: "))

arr = [] print("Enter elements: ")

for i in range(n):

    arr.append(int(input()))

#Sorting the array

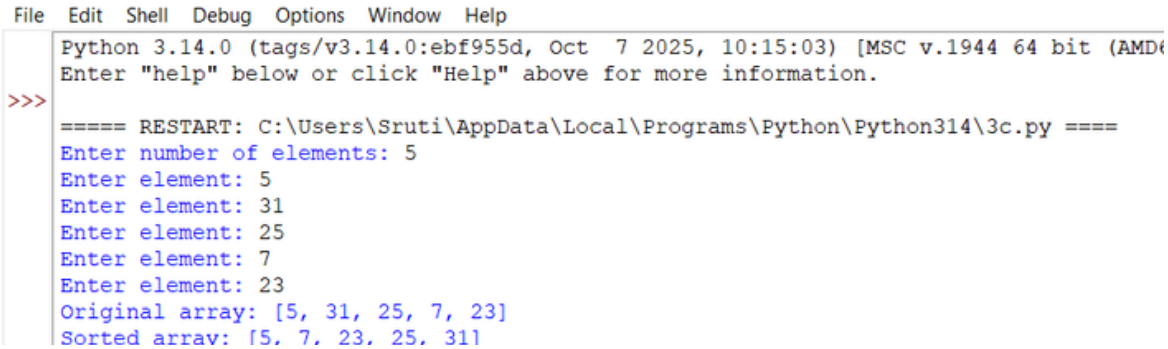
selection_sort(arr)

print("Sorted array: ")

for element in arr:

        print(element, end=" ")
```

OUTPUT:



EXPLANATION:

- The selection_sort() function is defined to arrange the elements of the list in **ascending order**.
- The outer for loop selects the position where the smallest element should be placed.
- The variable min_index stores the index of the smallest element found in the unsorted part of the list.
- The inner for loop compares the elements and updates min_index whenever a smaller element is found.
- The smallest element is swapped with the element at position i, and after all passes, the sorted array is displayed.

ERROR TABLE:

S.no.	Errors	Rectification
1.	**IndentationError** if the function or loop statements are not properly indented.	Maintain proper indentation inside the function and loops.
2.	**ValueError** if a non-integer value is entered as an array element.	Enter only valid integer values.
3.	**NameError** if variables such as arr, i, j, or min_index are misspelled.	Check the variable names and use the correct spelling.
4.	**IndexError** if an invalid list index is accessed.	Use the correct loop ranges and ensure the index is within the list range.

Week-4

A.AIM: Write a python program to implement merge sort, where the input elements are taken from the user.

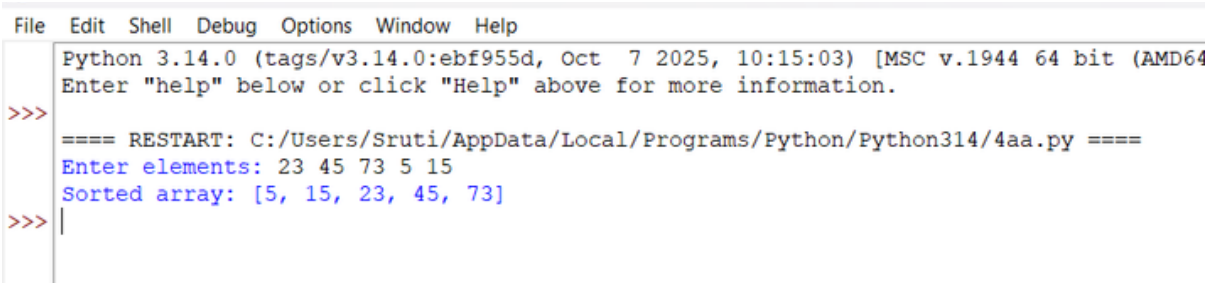
PROGRAM:

```
def merge_sort(arr):

    if len(arr) <= 1:
        return arr
    mid = len(arr) // 2
    left = arr[:mid]
    right = arr[mid:]
    left = merge_sort(left)
    right = merge_sort(right)
    i = j = k = 0
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            arr[k] = left[i]
            i += 1
        else:
            arr[k] = right[j]
            j += 1
        k += 1
    while i < len(left):
        arr[k] = left[i]
        i += 1
        k += 1
    while j < len(right):
        arr[k] = right[j]
        j += 1
```

```
        k += 1
    return arr
n = int(input("Enter number of elements: "))
arr = []
print("Enter elements:")
for i in range(n):
    arr.append(int(input()))
merge_sort(arr)
print("Sorted array:")
for element in arr:
    print(element, end=" ")
```

OUTPUT:



EXPLANATION:

- The merge_sort() function is defined to arrange the elements of the list in **ascending order** using the Merge Sort algorithm.
- The if len(arr) <= 1 condition checks whether the list has **one or no elements**. If so, the list is already sorted and returned.
- The variable mid stores the **middle index** of the list, which is used to divide the list into two halves.
- The list is divided into two parts: the left contains the first half and the right contains the second half.
- The merge_sort() function is called recursively to **sortboththeleftandrighthalves** .
- The variables i, j, and k are used as indexes for the **left list, right list, and original list** respectively.
- The while loop compares the elements of the left and right lists and places the **smaller element** into the original list.
- The next while loops copy any **remaining elements** from the left or right list into the original list.

- The return arr statement returns the **sorted list** after merging the two halves.
- The user enters the **number of elements** and the elements of the list using input().
- The merge_sort(arr) function is called to **sort the given list**.
- Finally, the sorted elements are displayed using a for loop.

ERROR TABLE:

S.no.	Errors	Rectification
1.	**IndentationError** occurred because some statements inside the function and loops were not properly aligned.	Corrected the indentation of the statements according to Python syntax.
2.	**ValueError** occurred when I entered a non-numeric value while giving the array elements.	Entered only integer values as input for the array.
3.	**TypeError** occurred because incompatible data types were present while comparing the elements.	Used integer values of the same suitable data type for sorting.
4.	**Recursion Error** occurred because the recursive function did not reach the stopping condition properly.	Added and verified the base condition if len(arr) <= 1.
5.	**IndexError** occurred when the program tried to access an element outside the valid range of the list.	Corrected the loop conditions and ensured that the indexes remain within the list range.

B.AIM: Write a python program to implement quick sort, where the input elements are taken from the user.

PROGRAM:

```
def quicksort(a, low, high):

    if low < high:
        i = low
        j = high
        pivot = low
        while i < j:
            while i <= high and a[i] <= a[pivot]:
                i += 1
            while a[j] > a[pivot]:
                j -= 1
            if i < j:
                a[i], a[j] = a[j], a[i]
        a[pivot], a[j] = a[j], a[pivot]
            quicksort(a, low, j - 1)
            quicksort(a, j + 1, high)
 n = int(input("Enter number of elements: "))
a = []
print("Enter elements:")
for i in range(n):
    a.append(int(input()))
quicksort(a, 0, n - 1)
print("Sorted array:")
for element in a:
    print(element, end=" ")
```

OUTPUT:

```
Python 3.14.0 (tags/v3.14.0:ebf955d, Oct  7 2025, 10:15:03) [MSC v.1944 64 bit (AM
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:/Users/Sruti/AppData/Local/Programs/Python/Python314/4b.py ====
Enter numbers to sort: 23 36 5 31 25
Sorted array:
[5, 23, 25, 31, 36]
```

EXPLANATION:

- The quicksort() function is used to arrange the elements of the array in **ascending order**.
- The condition if low < high checks whether the array portion contains more than one element.
- The variables i and j are used to **scan the array from both sides**.
- The variable pivot stores the index of the **first element selected as the pivot**.
- The first while loop moves i forward until an element greater than the pivot is found.
- The second while loop moves j backward until an element smaller than or equal to the pivot is found.
- If i < j, the elements at i and j are **swapped**.
- The pivot is then swapped with the element at j, placing it in its **correct position**.
- The function is recursively called for the **left and right portions** of the pivot.
- The user enters the number of elements and their values, and quicksort() is called to sort the array.
- Finally, the **sorted array is displayed**.

ERROR TABLE:

S.no.	Errors	Rectification
1.	**ValueError** occurred when I entered a non-integer value for an array element.	Entered only valid integer values while providing the input.
2.	**NameError** occurred because I used an incorrect variable name such as pivot, i, or j.	Checked the variable names and corrected their spelling throughout the program.
3.	**Index Error** occurred when the program tried to access an array position outside its valid range.	Corrected the loop conditions and ensured that the indexes remain within the array boundaries.
4.	**Recursion Error** occurred when the recursive calls continued without reaching the proper stopping condition.	Ensured that the condition if low < high correctly controls the recursive calls.

Week-5

A.AIM: Writeapythonprogramtoimplement single linked list with below operations

1. Create Linked list
2. Insert beginning
3. Insert end
4. Insert at index
5. Delete by value
6. Delete first node
7. Delete last node
8. Count number of nodes
9. Display/Traverse
10. Exit

PROGRAM:

class Node: def **init**(self, data): self.data = data self.next = None
class SinglyLinkedList: def **init**(self): self.head = None

```
# 1. Create linked list
def create(self):

    n = int(input("Enter number of nodes: "))

    for i in range(n):

        data = int(input(f"Enter data for node {i + 1}: "))
        new_node = Node(data)

        if self.head is None:
            self.head = new_node
        else:
            temp = self.head
            while temp.next:
                temp = temp.next
            temp.next = new_node

    print("Linked list created successfully.")

# 2. Insert at beginning
```

```
def insert_beginning(self):
    data = int(input("Enter data: "))
    new_node = Node(data)

    new_node.next = self.head
    self.head = new_node

    print("Node inserted at beginning.")

# 3. Insert at end
def insert_end(self):
    data = int(input("Enter data: "))
    new_node = Node(data)

    if self.head is None:
        self.head = new_node
    else:
        temp = self.head
            while temp.next:

                temp = temp.next

            temp.next = new_node

         print("Node inserted at end.")

     # 4. Insert at specific index

    def insert_index(self):

        index = int(input("Enter index: "))

        data = int(input("Enter data: "))

        new_node = Node(data)

         if index == 0:

            new_node.next = self.head

            self.head = new_node

            print("Node inserted.")

            return
```

```
        temp= self.head
        foriin range(index - 1):
            iftemp is None:
                print("Invalid index.")
                return
            temp = temp.next
        iftemp is None:
            print("Invalid index.")
            return
        new_node.next = temp.next
       temp.next = new_node
        print("Node inserted.")
    #5.Delete by value
    defdelete_value(self):
       value= int(input("Enter value to delete: "))
        ifself.head is None:
            print("List is empty.")
            return
        ifself.head.data == value:
            self.head = self.head.next
            print("Node deleted.")
            return
        temp= self.head

       whiletemp.next and temp.next.data != value:
```

```
        temp = temp.next
      iftemp.next is None:
        print("Value not found.")
    else:
        temp.next = temp.next.next
        print("Node deleted.")
 #6.Delete first node
defdelete_first(self):
    ifself.head is None:
        print("List is empty.")
    else:
        self.head = self.head.next
        print("First node deleted.")
 #7.Delete last node
defdelete_last(self):
    ifself.head is None:
        print("List is empty.")
        return
     ifself.head.next is None:
        self.head = None
        print("Last node deleted.")
        return

    temp = self.head
     while temp.next.next:
```

```
            temp = temp.next
        temp.next = None
        print("Last node deleted.")
    # 8. Count nodes
    def count(self):
        count = 0
        temp = self.head
        while temp:
            count += 1
            temp = temp.next
        print("Number of nodes:", count)
    # 9. Display / Traverse
    def display(self):
        if self.head is None:
            print("List is empty.")
            return
        temp = self.head
        while temp:
            print(temp.data, end=" -> ")
            temp = temp.next
        print("None")
# Main program
sll = SinglyLinkedList()
while True:
    print("\n--- SINGLY LINKED LIST ---")
```

```
print("1. Create linked list")
print("2. Insert at beginning")
print("3. Insert at end")
print("4. Insert at specific index")
print("5. Delete by value")
print("6. Delete first node")
print("7. Delete last node")
print("8. Count number of nodes")
print("9. Display / Traverse")
print("10. Exit")
 choice = int(input("Enter your choice: "))
 ifchoice == 1:
   sll.create()
 elif choice == 2:
   sll.insert_beginning()
 elif choice == 3:
   sll.insert_end()
 elif choice == 4:
   sll.insert_index()
 elif choice == 5:
   sll.delete_value()
 elif choice == 6:
   sll.delete_first()
 elif choice == 7:
   sll.delete_last()
```

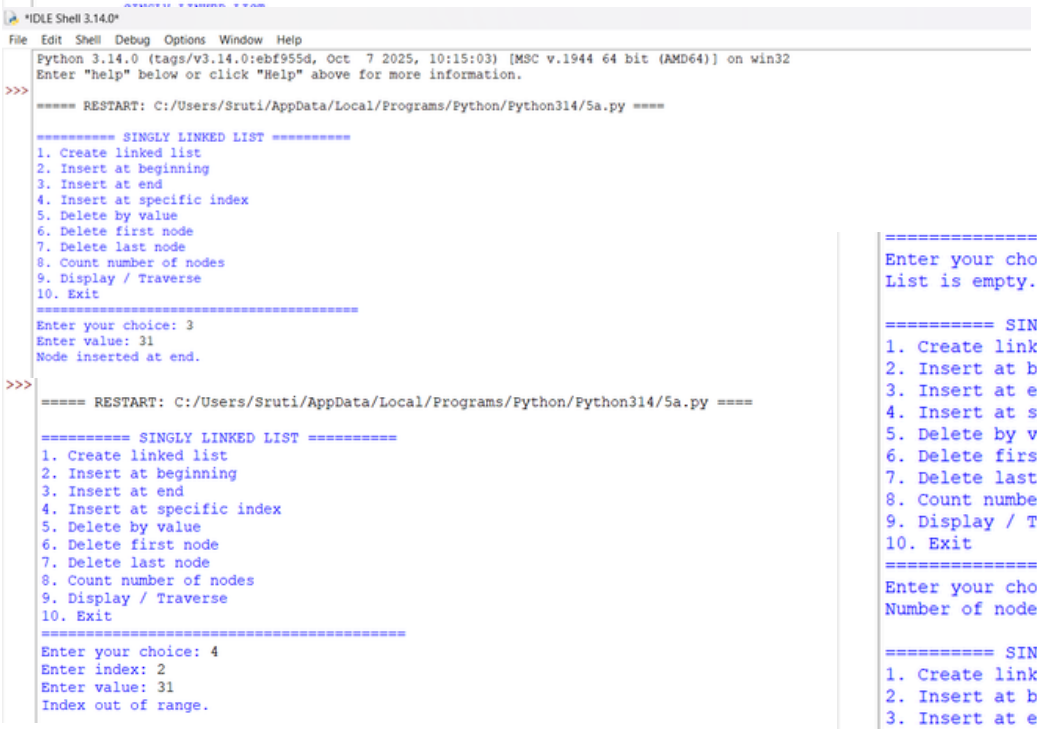
```
elif choice == 8:
    sll.count()
elif choice == 9:
    sll.display()
elif choice == 10:
    print("Program exited.")
    break
else:
    print("Invalid choice.")
```

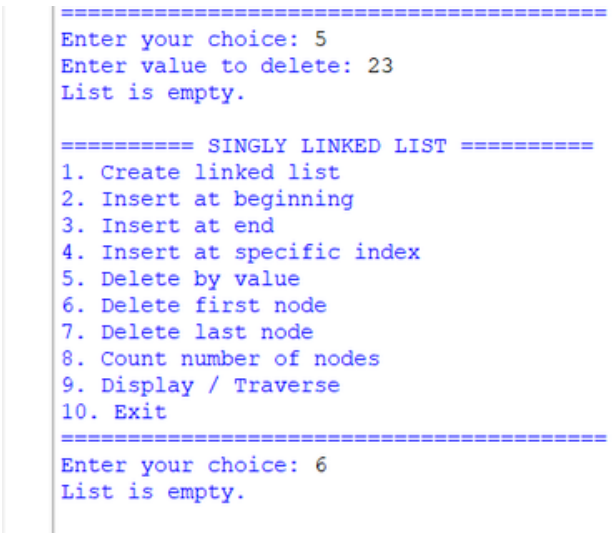
OUTPUT:

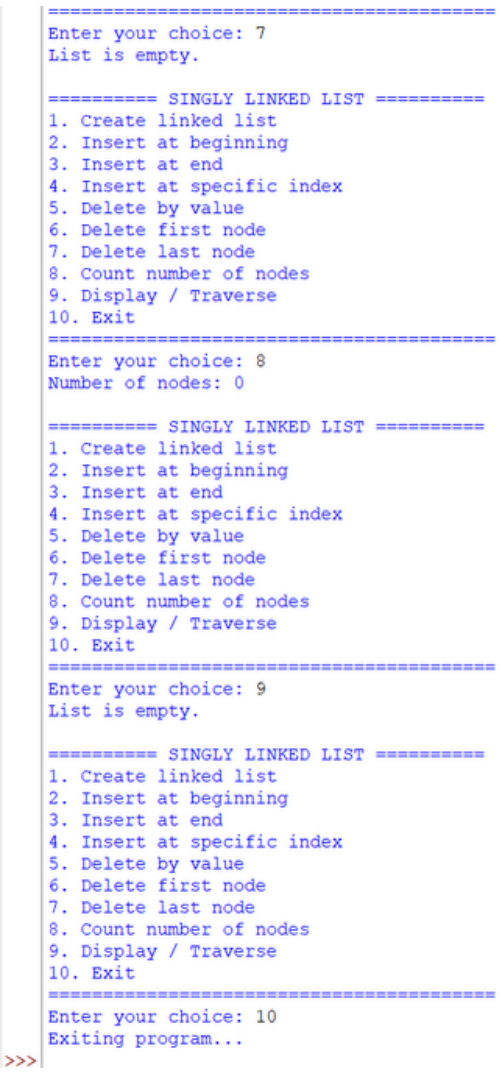
```
7. Delete last node
8. Count number of nodes
9. Display / Traverse
10. Exit
========================================
Enter your choice: 11
Invalid choice. Please try again.

========== SINGLY LINKED LIST ==========
1. Create linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete first node
7. Delete last node
8. Count number of nodes
9. Display / Traverse
10. Exit
========================================
Enter your choice: 1
Enter number of nodes: 5
Enter data for node 1: 5
Enter data for node 2: 31
Enter data for node 3: 23
Enter data for node 4: 25
Enter data for node 5: 7
Linked list created successfully.
```









EXPLANATION:

- The Node class is used to create a **nodecontainingdataandalinktothenext node** .
- The SinglyLinkedList class maintains the linked list using the head pointer.

- The create() function creates a linked list by adding the given number of nodes.
- The insert_beginning() function adds a **newnodeat thebeginning** of the list.
- The insert_end() function adds a **newnodeat theend** of the list.
- The insert_index() function inserts a node at a **specified index**.
- The delete_value() function searches for a given value and **deletesthecorresponding node**.
- The delete_first() function removes the **first node** from the list.
- The delete_last() function removes the **last node** from the list.
- The count() function counts and displays the **totalnumberof nodes** .
- The display() function traverses the list and **prints alltheelements** .
- The main program displays a **menu** and performs the selected linked-list operation.

ERROR TABLE:

S.no.	Errors	Rectification
1.	**Node Error** occurred because the Node class was not defined properly.	Defined the Node class before using it.
2.	**Attribute Error** occurred when temp.next was not handled correctly.	Updated temp using temp = temp.next.
3.	**Index Error** occurred when an invalid index was entered.	Checked whether the index is within the list range.
4.	**Value not found** occurred when the entered value was not present in the list.	Searched the list and displayed **"Valuenot found."**
5.	**Empty list error** occurred when an operation was performed on an empty list.	Checked self.head is None before performing the operation.

B.AIM: Write a Python program to implement a Double linked list with the above operations.

PROGRAM:

```
classNode:def__init__(self,data): self.data = data self.prev = None self.next = None
classDoublyLinkedList:def__init__(self): self.head = None

#1.Createlinkedlist
defcreate(self):

    n=int(input("Enternumber of nodes: "))

    foriinrange(n):

        data=int(input(f"Enterdata for node {i + 1}: "))
        new_node=Node(data)
        ifself.headisNone:
            self.head=new_node
        else:
            temp=self.head
            whiletemp.next:
                temp=temp.next
             temp.next=new_node
             new_node.prev=temp
    print("Doublylinkedlistcreated successfully.")


#2.Insertatbeginning
definsert_beginning(self):
    data=int(input("Enterdata: "))
    new_node=Node(data)
    ifself.headisnotNone:
        new_node.next=self.head
        self.head.prev=new_node
    self.head=new_node
    print("Nodeinsertedatbeginning.")


#3.Insertatend
definsert_end(self):
    data=int(input("Enterdata: "))
    new_node=Node(data)
    ifself.headisNone:
        self.head=new_node
    else:

        temp=self.head
        whiletemp.next:
```

```
            temp = temp.next
            temp.next=new_node
            new_node.prev=temp
            print("Nodeinserted at end.")


# 4. Insert at specific index
def insert_index(self):
    index=int(input("Enterindex: "))
    data=int(input("Enterdata: "))
    new_node = Node(data)
    if index == 0:
        if self.head:
            new_node.next=self.head
            self.head.prev=new_node
        self.head=new_node
        print("Nodeinserted.")
        return
    temp = self.head
    foriinrange(index-1):
        if temp is None:
            print("Invalidindex.")
            return
        temp = temp.next
    if temp is None:
        print("Invalid index.")
        return
    new_node.next=temp.next
    new_node.prev = temp
    if temp.next:
        temp.next.prev=new_node
    temp.next = new_node
    print("Node inserted.")


# 5. Delete by value
def delete_value(self):
    value=int(input("Entervalue to delete: "))
    temp = self.head
    whiletempandtemp.data != value:
        temp = temp.next
    if temp is None:
        print("Valuenotfound.")
        return
```

```
 if temp.prev:
        temp.prev.next = temp.next
    else:
        self.head = temp.next

    if temp.next:
        temp.next.prev = temp.prev

    print("Node deleted.")

# 6. Delete first node
def delete_first(self):
    if self.head is None:
        print("List is empty.")
        return

    self.head = self.head.next

    if self.head:
         self.head.prev = None

    print("First node deleted.")

# 7. Delete last node
def delete_last(self):
    if self.head is None:
        print("List is empty.")
        return

    temp = self.head

    while temp.next:
        temp = temp.next

    if temp.prev:
        temp.prev.next = None
    else:
        self.head = None

    print("Last node deleted.")

# 8. Count nodes
```

```
defcount(self):
    count = 0
    temp = self.head

    while temp:
        count += 1
        temp = temp.next

    print("Number of nodes:", count)

#9.Display / Traverse
defdisplay(self):
    ifself.head is None:
        print("List is empty.")
        return
    temp = self.head
    while temp:
        print(temp.data, end=" <-> ")
        temp = temp.next

    print("None")


#Main program

dll = DoublyLinkedList()

while True:

    print("\n--- DOUBLY LINKED LIST ---")

    print("1. Create linked list")

    print("2. Insert at beginning")

    print("3. Insert at end")

    print("4. Insert at specific index")

    print("5. Delete by value")

    print("6. Delete first node")

    print("7. Delete last node")

    print("8. Count number of nodes")

    print("9. Display / Traverse")
```

```
    print("10. Exit")

    choice = int(input("Enter your choice: "))

ifchoice == 1:
    dll.create()
elifchoice == 2:
    dll.insert_beginning()
elifchoice == 3:
    dll.insert_end()
elifchoice == 4:
    dll.insert_index()
elifchoice == 5:
    dll.delete_value()
elifchoice == 6:
    dll.delete_first()
elifchoice == 7:
    dll.delete_last()
elifchoice == 8:
    dll.count()
elifchoice == 9:
    dll.display()
elifchoice == 10:
    print("Program exited.")
    break
else:
    print("Invalid choice.")
```

OUTPUT:

```
>>>
===== RESTART: C:/Users/Sruti/AppData/Local/Programs/Python/Python314/5b.py ====

========== DOUBLY LINKED LIST ==========
1. Create linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete first node
7. Delete last node
8. Count number of nodes
9. Display / Traverse
10. Exit
========================================
Enter your choice: 1
Enter number of nodes: 4
Enter data for node 1: 23
Enter data for node 2: 25
Enter data for node 3: 31
Enter data for node 4: 5
Doubly linked list created successfully.

========== DOUBLY LINKED LIST ==========
1. Create linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete first node
7. Delete last node
8. Count number of nodes
9. Display / Traverse
10. Exit
========================================
Enter your choice: 2
Enter value: 23
Node inserted at beginning.

========================================
Enter your choice: 3
Enter value: 5
Node inserted at end.

========== DOUBLY LINKED LIST ==========
1. Create linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete first node
7. Delete last node
8. Count number of nodes
9. Display / Traverse
10. Exit
========================================
Enter your choice: 4
Enter index: 3
Enter value: 25
Node inserted successfully.

========== DOUBLY LINKED LIST ==========
1. Create linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete first node
7. Delete last node
8. Count number of nodes
9. Display / Traverse
10. Exit
========================================
Enter your choice: 5
Enter value to delete: 23
Node deleted successfully.

========== DOUBLY LINKED LIST ==========
1. Create linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete first node
7. Delete last node
8. Count number of nodes
9. Display / Traverse
10. Exit
========================================
Python 3.14.0 (tags/v3.14.0:ebf955d, Oct  7 2025, 10:15:03) [MSC v.1944 64 bit (AMD64)] on wi
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:/Users/Sruti/AppData/Local/Programs/Python/Python314/5b.py ====

========== DOUBLY LINKED LIST ==========
1. Create linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete first node
7. Delete last node
8. Count number of nodes
9. Display / Traverse
10. Exit
========================================
Enter your choice: 6
List is empty.
```

```
Invalid choice. Please try again.

========== DOUBLY LINKED LIST ==========
1. Create linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete first node
7. Delete last node
8. Count number of nodes
9. Display / Traverse
10. Exit
========================================
Enter your choice: 10
```

EXPLANATION:

- The Node class creates a node containing **data, previous link, and next link**.
- The DoublyLinkedList class maintains the list using the head pointer.
- The create() function creates the doubly linked list by adding the given nodes.
- The insert_beginning() function adds a **new node at the beginning** and updates the previous link.
- The insert_end() function adds a **new node at the end** of the list.
- The insert_index() function inserts a node at a **specified index** and updates both links.
- The delete_value() function searches for a given value and **removes that node**.
- The delete_first() function removes the **first node** and updates the head.
- The delete_last() function removes the **last node** from the list.
- The count() function counts and displays the **number of nodes**.
- The display() function traverses the list and displays the elements using **<->**.
- The mainprogramprovides a **menu to select and perform different operations**.

ERROR TABLE:

S.no.	Errors	Rectification
1.	**IndentationError** occurred because the statements were not properly aligned.	Corrected the indentation of all blocks.
2.	**NameError** occurred because new_node, temp, or other variables were misspelled.	Corrected the variable names and spelling.
3.	**AttributeError** occurred when prev or next links were not updated correctly.	Updated the prev and next links properly.
4.	**IndexError** occurred when an invalid index was entered.	Checked whether the index is within the valid range.

C.AIM: Write a Python program to implement a Circular linked list with the above operations.

PROGRAM:

```
class Node:


        def init(self, data):

             self.data = data self.next = None

class CircularLinkedList:

       def init(self):

            self.head = None

# 1. Create linked list
def create(self):
   n = int(input("Enter number of nodes: "))
   for i in range(n):

       data = int(input(f"Enter data for node {i + 1}: "))
       new_node = Node(data)

       if self.head is None:
          self.head = new_node
          new_node.next = self.head
       else:
          temp = self.head

          while temp.next != self.head:
          temp = temp.next

          temp.next = new_node
          new_node.next = self.head

   print("Circular linked list created successfully.")

# 2. Insert at beginning
def insert_beginning(self):
   data = int(input("Enter data: "))
   new_node = Node(data)
```

```
    if self.head is None:
        self.head = new_node
        new_node.next = self.head
    else:
        temp = self.head

        while temp.next != self.head:
            temp = temp.next

        new_node.next = self.head
        temp.next = new_node
        self.head = new_node

    print("Node inserted at beginning.")

# 3. Insert at end
def insert_end(self):
    data = int(input("Enter data: "))
    new_node = Node(data)

    if self.head is None:
        self.head = new_node
        new_node.next = self.head
    else:
        temp = self.head

        while temp.next != self.head:
        temp = temp.next

        temp.next = new_node
        new_node.next = self.head

    print("Node inserted at end.")

# 4. Insert at specific index
def insert_index(self):
    index = int(input("Enter index: "))
    data = int(input("Enter data: "))
    new_node = Node(data)

    if index == 0:
        if self.head is None:
            self.head = new_node
```

```
            new_node.next = self.head
        else:
            temp = self.head

            while temp.next != self.head:
                temp = temp.next

            new_node.next = self.head
            temp.next = new_node
            self.head = new_node

        print("Node inserted.")
        return

    if self.head is None:
        print("Invalid index.")
        return

    temp = self.head

    for i in range(index - 1):
        temp = temp.next

        if temp == self.head:
            print("Invalid index.")
            return

    new_node.next  =  temp.next
    temp.next = new_node

    print("Node inserted.")

# 5. Delete by value
def delete_value(self):
    value = int(input("Enter value to delete: "))

    if self.head is None:
        print("List is empty.")
        return

    # If head contains the value
    if self.head.data == value:
```

```
        if self.head.next == self.head:
            self.head = None
        else:
            temp = self.head

            while temp.next != self.head:
                temp = temp.next

            temp.next = self.head.next
            self.head = self.head.next

        print("Node deleted.")
        return

    temp = self.head

    while temp.next != self.head and temp.next.data != value:
        temp = temp.next

    if temp.next == self.head:
        print("Value not found.")
    else:
        temp.next = temp.next.next
        print("Node deleted.")

# 6. Delete first node
def delete_first(self):
    if self.head is None:
        print("List is empty.")
        return

    if self.head.next == self.head:
        self.head = None
    else:
        temp = self.head

        while temp.next != self.head:
            temp = temp.next

        temp.next = self.head.next
        self.head = self.head.next
```

```
    print("First node deleted.")

# 7. Delete last node
def delete_last(self):
    if self.head is None:
        print("List is empty.")
        return

    if self.head.next == self.head:
        self.head = None
    else:
        temp = self.head

        while temp.next.next != self.head:
            temp = temp.next

        temp.next = self.head

    print("Last node deleted.")

# 8. Count nodes
def count(self):
    if self.head is None:
        print("Number of nodes: 0")
        return

    count = 0
    temp = self.head

    while True:
        count += 1
        temp = temp.next

        if temp == self.head:
            break

    print("Number of nodes:", count)
# 9. Display / Traverse
def display(self):
    if self.head is None:
        print("List is empty.")
        return
```

```
 temp = self.head

    while True:
        print(temp.data, end=" -> ")
        temp = temp.next

        if temp == self.head:
            break

    print("HEAD")
 #Main program

cll = CircularLinkedList()

while True:

    print("\n--- CIRCULAR LINKED LIST ---")

    print("1. Create linked list")

    print("2. Insert at beginning")

    print("3. Insert at end")

    print("4. Insert at specific index")

    print("5. Delete by value")

    print("6. Delete first node")

    print("7. Delete last node")

    print("8. Count number of nodes")

    print("9. Display / Traverse")

    print("10. Exit")

choice = int(input("Enter your choice: "))

if choice == 1:
    cll.create()
elif choice == 2:
    cll.insert_beginning()
elif choice == 3:
```

```
        cll.insert_end()
    elif choice == 4:
        cll.insert_index()
    elif choice == 5:
        cll.delete_value()
    elif choice == 6:
        cll.delete_first()
    elif choice == 7:
        cll.delete_last()
    elif choice == 8:
        cll.count()

    elif choice == 9:
        cll.display()

    elif choice == 10:
        print("Program exited.")
        break

    else:
        print("Invalid choice.")
```

OUTPUT:

```
6. Delete first node
7. Delete last node
8. Count number of nodes
9. Display / Traverse
10. Exit
========================================
Enter your choice: 1
Enter number of nodes: 4
Enter data for node 1: 31
Enter data for node 2: 5
Enter data for node 3: 23
Enter data for node 4: 25
Circular linked list created successfully.

========== CIRCULAR LINKED LIST ==========
1. Create linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete first node
7. Delete last node
```

```
==========================================
Enter your choice: 2
Enter value: 31
Node inserted at beginning.

========== CIRCULAR LINKED LIST ==========
1. Create linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete first node
7. Delete last node
8. Count number of nodes
9. Display / Traverse
10. Exit
==========================================
Enter your choice: 3
Enter value: 23
Node inserted at end.
```

```
>>>
===== RESTART: C:/Users/Sruti/AppData/Local/Programs/Python/Python314/5c.py =====

========== CIRCULAR LINKED LIST ==========
1. Create linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete first node
7. Delete last node
8. Count number of nodes
9. Display / Traverse
10. Exit
==========================================
Enter your choice: 4
Enter index: 3
Enter value: 23
Index out of range.

========== CIRCULAR LINKED LIST ==========
1. Create linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete first node
7. Delete last node
8. Count number of nodes
9. Display / Traverse
10. Exit
==========================================
Enter your choice: 5
Enter value to delete: 31
List is empty.
```

```
Enter your choice: 6
List is empty.

========== CIRCULAR LINKED LIST ==========
1. Create linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete first node
7. Delete last node
8. Count number of nodes
9. Display / Traverse
10. Exit
==========================================
Enter your choice: 8
Number of nodes: 0

========== CIRCULAR LINKED LIST ==========
1. Create linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete first node
7. Delete last node
8. Count number of nodes
9. Display / Traverse
10. Exit
==========================================
Enter your choice: 10
Exiting program...
>>>
```

EXPLANATION:

- The Node class creates a node containing **dataand alink tothenext node** .
- The CircularLinkedList class maintains the list using the head pointer.

- The create() function creates the circular linked list and connects the **last node back to the head**.
- The insert_beginning() function adds a **new node at the beginning** and updates the circular connection.
- The insert_end() function adds a **new node at the end** and connects it back to the head.
- The insert_index() function inserts a node at a **specified index**.
- The delete_value() function searches for a value and **deletes the corresponding node**.
- The delete_first() function removes the **first node** and updates the head.
- The delete_last() function removes the **last node** while maintaining the circular connection.
- The count() function counts and displays the **total number of nodes**.
- The display() function traverses the list and displays the elements until it reaches the **head again**.
- The main program provides a **menu to perform different circular linked-list operations**.

ERROR TABLE:

S.no.	Errors	Rectification
1.	**IndentationError** occurred because the statements were not properly aligned.	Corrected the indentation of all code blocks.
2.	**NameError** occurred because variables like temp or new_node were misspelled.	Corrected the variable names and spelling.
3.	**AttributeError** occurred when the next link was not updated properly.	Updated the next links correctly to maintain the circular connection.
4.	**IndexError** occurred when an invalid index was entered.	Checked whether the index is within the valid range.
5.	**Value not found** occurred when the entered value was not present in the list.	Searched the list and displayed **"Valuenot found."**

Week-6

A.AIM: Write a Python program to implement stack operations such as Push, Pop, Peek, Display using an array

PROGRAM:

```
stack = []

 # Push operation

def push():

    value = int(input("Enter element: "))

    stack.append(value)

    print("Element pushed.")

 # Pop operation

def pop():

    if len(stack) == 0:

        print("Stack is empty.")

    else:

        print("Popped element:", stack.pop())

 # Peek operation

def peek():

    if len(stack) == 0:

        print("Stack is empty.")

    else:

        print("Top element:", stack[-1])

 # Display operation

def display():

    if len(stack) == 0:

        print("Stack is empty.")

    else:

        print("Stack elements:")
```

```
        for i in range(len(stack) - 1, -1, -1):
            print(stack[i])
 # Main program
while True:
    print("\n--- STACK USING ARRAY ---")
    print("1. Push")
    print("2. Pop")
    print("3. Peek")
    print("4. Display")
    print("5. Exit")
    choice = int(input("Enter your choice: "))
     if choice == 1:
        push()
     elif choice == 2:
        pop()
     elif choice == 3:
        peek()
     elif choice == 4:
        display()
     elif choice == 5:
       print("Program exited.")
        break
     else:
        print("Invalid choice.")
```

OUTPUT:

```
>>>
===== RESTART: C:/Users/Sruti/AppData/Local/Programs/Python/Python314/6a.py ====

1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter element to push: 2
Element pushed successfully.

1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 2
Popped element: 2

1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 3
Stack is empty

1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 4
Stack is empty

1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 5
>>>
>>>
```

```
>>>
===== RESTART: C:/Users/Sruti/AppData/Local/Programs/Python/Python314/6a.py ====

1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 7
Invalid choice

1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice:
```

EXPLANATION:

- The stack list is used to store the **stack elements**.
- The push() function adds an element to the **top of the stack**.
- The pop() function removes and returns the **top element**.
- The peek() function displays the **top element without removing it**.
- The display() function prints all elements from **top to bottom**.
- len(stack) == 0 checks whether the stack is **empty**.
- The main program provides a **menu** to perform different stack operations.
- Stack follows the **LIFO (Last In, First Out)** principle.

ERROR TABLE:

S.no.	Errors	Rectification
1.	**ValueError** occurred when a non-integer value was entered.	Entered only valid integer values.
2.	**NameError** occurred because stack or a function name was misspelled.	Corrected the variable and function names.
3.	**Index Error** occurred when trying to access the top element of an empty stack.	Checked whether the stack is empty before accessing an element.
4.	**Empty stack error** occurred when Pop or Peek was performed without any elements.	Checked len(stack) == 0 before performing Pop or Peek.
5.	**SyntaxError** occurred because of incorrect Python syntax.	Checked and corrected the syntax of the program.

B.AIM: Write a Python program to implement stack operations such as Push, Pop, Peek, Display using a linked list

PROGRAM:

```
class Node:

    def init(self, data):

        self.data = data self.next = None
```

```
classStack:

    def__init__(self):

        self.top = None

#Pushoperation
defpush(self):
    value= int(input("Enter element: "))

    new_node        =        Node(value)
    new_node.next = self.top
    self.top = new_node

    print("Element pushed.")

#Popoperation
defpop(self):
    ifself.top is None:
        print("Stack is empty.")
    else:
        print("Popped element:", self.top.data)
        self.top = self.top.next

#Peekoperation
defpeek(self):
    ifself.top is None:
        print("Stack is empty.")
    else:
        print("Top element:", self.top.data)

#Display operation
defdisplay(self):
    ifself.top is None:
        print("Stack is empty.")
    else:
        temp = self.top

        print("Stack elements:")
        while temp:
            print(temp.data)
            temp = temp.next
```

```
#Main program

stack = Stack()

while True:

    print("\n--- STACK USING LINKED LIST ---")

    print("1. Push")

    print("2. Pop")

    print("3. Peek")

    print("4. Display")

    print("5. Exit")

choice = int(input("Enter your choice: "))

if choice == 1:
stack.push()

elif choice == 2:
    stack.pop()

elif choice == 3:
    stack.peek()

elif choice == 4:
    stack.display()

elif choice == 5:
    print("Program exited.")
    break

else:
    print("Invalid choice.")
```

OUTPUT:

```
>>>
===== RESTART: C:/Users/Sruti/AppData/Local/Programs/Python/Python314/6b.py ====

1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter element: 7
Element pushed

1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 2
Popped element: 7

1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 3
Stack is empty

1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice:
```

```
>>>
===== RESTART: C:/Users/Sruti/AppData/Local/Programs/Python/Python314/6b.py ====

1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 4
Stack is empty

1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 5
>>>
```

EXPLANATION:

- The Node class creates a node containing **dataand alink tothenext node** .
- The Stack class uses top to keep track of the **top element**.
- The push() function adds a new node at the **top of thestack** .
- The pop() function removes the **top node** from the stack.
- The peek() function displays the **top element without removing it**.
- The display() function traverses the linked list and displays all elements.
- self.top is None checks whether the stack is **empty**.
- The main program provides a **menu** for performing stack operations.
- The stack follows the **LIFO(Last In,First Out)** principle.

ERROR TABLE:

S.no.	Errors	Rectification
1.	**NameError** occurred because top, temp, or new_node was misspelled.	Corrected the variable names and their spelling.
2.	**AttributeError** occurred when next was accessed incorrectly.	Correctly used self.top and temp.next.
3.	**ValueError** occurred when a non-integer value was entered.	Entered only valid integer values.
4.	**Empty stack error** occurred when Pop or Peek was performed without any elements.	Checked self.top is None before performing the operation.
5.	**IndentationError** occurred because the statements inside the functions were not properly aligned.	Corrected the indentation of all code blocks.

Week-7

A.AIM: Write a Python program to implement a queue and its operations such as Enqueue,Dequeue, Peek, Display with a menu-driven approach. Implement using array

PROGRAM:

```
queue = []

# Enqueue operation
```

```
def enqueue():
    value = int(input("Enter element: "))
    queue.append(value)
    print("Element inserted.")
# Dequeue operation
def dequeue():
    if len(queue) == 0:
        print("Queue is empty.")
    else:
        print("Deleted element:", queue.pop(0))
# Peek operation
def peek():
    if len(queue) == 0:
        print("Queue is empty.")
    else:
        print("Front element:", queue[0])
# Display operation
def display():
    if len(queue) == 0:
        print("Queue is empty.")
    else:
        print("Queue elements:")
        for element in queue:
            print(element, end=" ")
        print()
```

```
# Main program
while True:
    print("\n--- QUEUE USING ARRAY ---")
    print("1. Enqueue")
    print("2. Dequeue")
    print("3. Peek")
    print("4. Display")
    print("5. Exit")
     choice = int(input("Enter your choice: "))
     if choice == 1:
        enqueue()
     elif choice == 2:
        dequeue()
     elif choice == 3:
        peek()
     elif choice == 4:
        display()
     elif choice == 5:
        print("Program exited.")
        break
     else:
        print("Invalid choice.")
```

OUTPUT:

```
>>>
===== RESTART: C:/Users/Sruti/AppData/Local/Programs/Python/Python314/7a.py ====

1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter element: 7
Element inserted

1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 2
Deleted element: 7

1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 3
Queue is empty
```

```
>>>
===== RESTART: C:/Users/Sruti/AppData/Local/Programs/Python/Python314/7a.py ====

1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 4
Queue is empty

1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 6
Invalid choice

1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 5
>>>
```

EXPLANATION:

- The queue list is used to store the **queue elements**.
- The enqueue() function adds an element at the **rear** of the queue.
- The dequeue() function removes an element from the **front** of the queue.
- The peek() function displays the **front element** without removing it.
- The display() function prints all the elements in the queue.
- len(queue) == 0 checks whether the queue is **empty**.
- The main program provides a **menu** to select different queue operations.
- Queue follows the **FIFO (First In, First Out)** principle.

ERROR TABLE:

S.no.	Errors	Rectification
1.	**IndentationError** occurred because the statements inside the functions were not properly aligned.	Corrected the indentation of all code blocks.
2.	**ValueError** occurred when a non-integer value was entered.	Entered only valid integer values.
3.	**NameError** occurred because queue or a function name was misspelled.	**NameError** occurred because queue or a function name was misspelled.
4.	**IndexError** occurred when trying to access an element from an empty queue.	Checked whether the queue is empty before accessing elements.
5.	**Empty queue error** occurred when Dequeue or Peek was performed without elements.	Checked len(queue) == 0 before performing the operation.

B.AIM:

Write a Python program to implement a queue and its operations such as Enqueue, Dequeue, Peek, Display with a menu-driven approach. Implement using Linked list data.

PROGRAM:

```
class Node:

    def init(self, data):

        self.data = data self.next = None

class Queue:

   def init(self):

        self.front = None self.rear = None

# Enqueue operation
def enqueue(self):
   value = int(input("Enter element: "))
   new_node = Node(value)

   if self.rear is None:
      self.front = self.rear = new_node
   else:
      self.rear.next = new_node
      self.rear = new_node
```

```
 print("Element inserted.")

# Dequeue operation
def dequeue(self):
    if self.front is None:
        print("Queue is empty.")
    else:
        print("Deleted element:", self.front.data)
        self.front = self.front.next

        if self.front is None:
            self.rear = None

# Peek operation
def peek(self):
    if self.front is None:
        print("Queue is empty.")
    else:
        print("Front element:", self.front.data)

# Display operation
def display(self):
    if self.front is None:
        print("Queue is empty.")
    else:
        temp = self.front

        print("Queue elements:")
        while temp:
            print(temp.data, end=" ")
            temp = temp.next

        print()
#Main program

queue = Queue()

while True:

    print("\n--- QUEUE USING LINKED LIST ---")

    print("1. Enqueue")
```

```
    print("2. Dequeue")

    print("3. Peek")

    print("4. Display")

    print("5. Exit")

choice = int(input("Enter your choice: "))

if choice == 1:
    queue.enqueue()
elif choice == 2:
    queue.dequeue()

elif choice == 3:
    queue.peek()
elif choice == 4:
    queue.display()
elif choice == 5:
    print("Program exited.")
    break
else:
    print("Invalid choice.")
```

OUTPUT:

```
===== RESTART: C:/Users/Sruti/AppData/Local/Programs/Python/Python314/7b.py ====

1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter element: 5
Element inserted

1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 2
Deleted element: 5

1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 3
Queue is empty
```

```
>>>
===== RESTART: C:/Users/Sruti/AppData/Local/Programs/Python/Python314/7b.py ====

1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 4
Queue is empty

1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 6
Invalid choice

1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 5
>>>
```

EXPLANATION:

- The Node class creates a node containing **data and a link to the next node**.
- The Queue class uses front and rear to maintain the queue.
- The enqueue() function adds a new node at the **rear**.
- The dequeue() function removes a node from the **front**.
- The peek() function displays the **front element** without removing it.
- The display() function traverses the linked list and displays all elements.
- The queue follows the **FIFO (First In, First Out)** principle.
- The main program provides a **menu-driven approach** for performing the operations.

ERROR TABLE:

S.no.	Errors	Rectification
1.	**NameError** occurred because front, rear, or new_node was misspelled.	Corrected the variable names and their spelling.
2.	**AttributeError** occurred when next was accessed incorrectly.	Used self.front, self.rear, and temp.next correctly.
3.	**ValueError** occurred when a non-integer value was entered.	Entered only valid integer values.
4.	**Empty queue error** occurred when Dequeue or Peek was performed without elements.	Checked self.front is None before performing the operation.
5.	**SyntaxError** occurred due to incorrect Python syntax.	Checked and corrected the syntax of the program.

C.AIM: Write a Python program to implement a queue and its operations such as Enqueue,Dequeue, Peek, Display with a menu-driven approach. Implement using Circular Queue array structures

PROGRAM:

```
class CircularQueue:


      def init(self, size):

            self.size = size self.queue = [None] * size self.front = -1 self.rear = -1

# Enqueue
def enqueue(self):
   value = int(input("Enter element: "))

  if (self.rear + 1) % self.size == self.front:
      print("Queue is full.")
      return

   if self.front == -1:
      self.front = 0

   self.rear = (self.rear + 1) % self.size
   self.queue[self.rear] = value

   print("Element inserted.")
```

```
# Dequeue
def dequeue(self):
    if self.front == -1:
        print("Queue is empty.")
        return

    print("Deleted element:", self.queue[self.front])

    if self.front == self.rear:
        self.front = -1
        self.rear = -1
    else:
        self.front = (self.front + 1) % self.size
# Peek
def peek(self):
    if self.front == -1:
        print("Queue is empty.")
    else:
        print("Front element:", self.queue[self.front])

# Display
def display(self):
    if self.front == -1:
        print("Queue is empty.")
        return

    print("Queue elements:")
    i = self.front

    while True:
        print(self.queue[i], end=" ")

        if i == self.rear:
            break

        i = (i + 1) % self.size

    print()
#Main program

size = int(input("Enter queue size: "))

queue = CircularQueue(size)
```

```
while True:
    print("\n--- CIRCULAR QUEUE ---")
    print("1. Enqueue")
    print("2. Dequeue")
    print("3. Peek")
    print("4. Display")
    print("5. Exit")

choice = int(input("Enter your choice: "))
if choice == 1:
    queue.enqueue()
elif choice == 2:
    queue.dequeue()
elif choice == 3:
    queue.peek()
elif choice == 4:
    queue.display()
elif choice == 5:
    print("Program exited.")
    break
else:
    print("Invalid choice.")
```

OUTPUT:

```
===== RESTART: C:/Users/Sruti/AppData/Local/Programs/Python/Python314/7c.py ====

1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter element: 5
Element inserted

1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 2
Deleted element: 5

1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 3
```

```
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 4
Queue is empty

1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 6
Invalid choice

1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
```

EXPLANATION:

- The CircularQueue class creates a **queue using an array**.
- front represents the position of the **first element**, while rear represents the **last element**.
- enqueue() adds a new element at the **rear**.
- dequeue() removes an element from the **front**.
- peek() displays the **front element** without deleting it.
- display() prints all the elements from **front to rear**.
- The % size operation makes the rear and front **wrap around the array**.
- The main program provides a **menu-driven interface**.
- The queue follows the **FIFO (First In, First Out)** principle.

ERROR TABLE:

S.no.	Errors	Rectification
1.	**IndentationError** occurred because the statements were not properly aligned.	Corrected the indentation of all code blocks.
2.	**ValueError** occurred when a non-integer value was entered.	Entered only valid integer values.
3.	**NameError** occurred because front, rear, or queue was misspelled.	Corrected the variable names and spelling.
4.	**IndexError** occurred when an invalid array position was accessed.	Used proper circular indexing with % size.
5.	**Queue full error** occurred when an element was inserted into a full queue.	Checked the full condition before enqueueing.

D.AIM: Write a Python program to implement a queue and its operations such as Enqueue,Dequeue, Peek, Display with a menu-driven approach. Implement using Circular Queue Linked list structures.

PROGRAM:

```
class Node:


    def init(self, data):

        self.data = data self.next = None

class CircularQueue:

    def init(self):

        self.front = None self.rear = None

# Enqueue
def enqueue(self):

    value = int(input("Enter element: "))
    new_node = Node(value)

    if self.front is None:
        self.front = self.rear = new_node
        new_node.next = self.front
    else:
    new_node.next = self.front
    self.rear.next = new_node
    self.rear = new_node

    print("Element inserted.")
# Dequeue
def dequeue(self):
    if self.front is None:
        print("Queue is empty.")
        return

    print("Deleted element:", self.front.data)

    if self.front == self.rear:
        self.front = None
        self.rear = None
    else:
```

```
        self.front = self.front.next
        self.rear.next= self.front

# Peek
def peek(self):
    ifself.frontisNone:
        print("Queueis empty.")
    else:
        print("Frontelement:", self.front.data)

# Display
def display(self):
    ifself.frontisNone:
        print("Queueis empty.")
        return

    temp=self.front

    print("Queueelements:")

    while True:
        print(temp.data, end=" ")

        temp=temp.next

       iftemp==self.front:
            break

    print()


#Main program

queue=CircularQueue()

while True:

    print("\n---CIRCULAR QUEUE USING LINKED LIST ---")

    print("1. Enqueue")

    print("2. Dequeue")

    print("3. Peek")
```

```
    print("4. Display")

    print("5. Exit")

choice = int(input("Enter your choice: "))
ifchoice == 1:
    queue.enqueue()
elifchoice == 2:
    queue.dequeue()
elifchoice == 3:
    queue.peek()
elifchoice == 4:
    queue.display()
elifchoice == 5:
    print("Program exited.")
    break
else:
    print("Invalid choice.")
```

OUTPUT:

```
>>>
    ===== RESTART: C:/Users/Sruti/AppData/Local/Programs/Python/Python314/7d.py ====

    1. Enqueue
    2. Dequeue
    3. Peek
    4. Display
    5. Exit
    Enter your choice: 1
    Enter element: 5
    Element inserted

    1. Enqueue
    2. Dequeue
    3. Peek
    4. Display
    5. Exit
    Enter your choice: 2
    Deleted element: 5

    1. Enqueue
    2. Dequeue
    3. Peek
    4. Display
    5. Exit
    Enter your choice: 3
    Queue is empty
```

```
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 4
Queue is empty

1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 6
Invalid choice

1. Enqueue
2. Dequeue
3. Peek
4. Display
```

EXPLANATION:

- The Node class creates a node containing **data and a next link**.
- The CircularQueue class maintains the queue using front and rear.
- The enqueue() function adds a node at the **rear** and connects it back to the front.
- The dequeue() function removes the **front node**.
- The peek() function displays the **front element** without removing it.
- The display() function traverses the circular linked list until it reaches the **front again**.
- front and rear are set to None when the queue becomes empty.
- The main program provides a **menu-driven approach**.
- The circular queue follows the **FIFO (First In, First Out)** principle.

ERROR TABLE:

S.no.	Errors	Rectification
1.	**NameError** occurred because front, rear, or new_node was misspelled.	Corrected the variable names and spelling.
2.	**AttributeError** occurred when the next link was not accessed correctly.	Used self.front, self.rear, and temp.next correctly.
3.	**ValueError** occurred when a non-integer value was entered.	Entered only valid integer values.
4.	**Empty queue error** occurred when Dequeue or Peek was performed on an empty queue.	Checked self.front is None before performing the operation.
5.	**Circular link error** occurred when the rear node was not connected back to the front node.	Updated self.rear.next = self.front after insertion and deletion.

LAB-8

1.AIM: Python program to implement a binary tree and its operations using an array and a linked list.

Program using array:

Week 8: Binary Tree using Array

```
tree = []

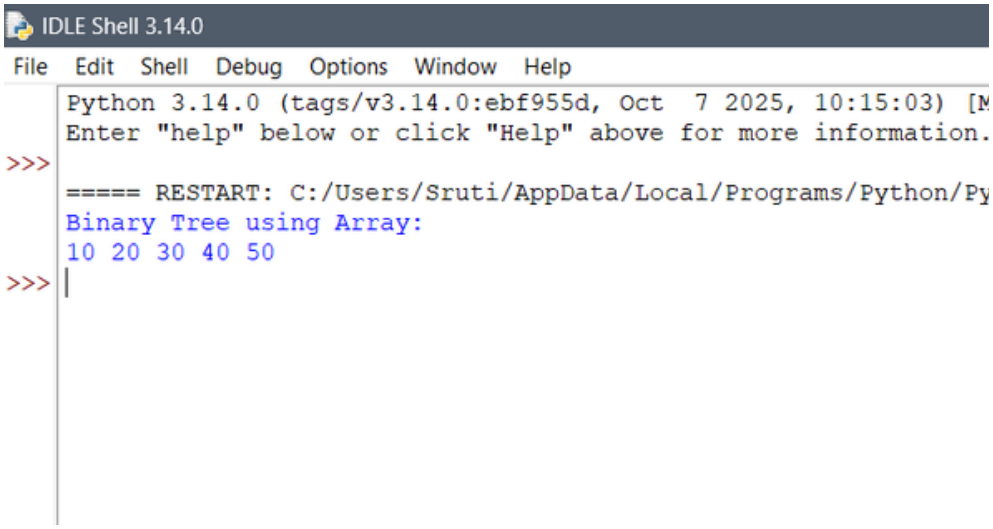
def insert(value):
    tree.append(value)
    print(value, "inserted successfully")

def display():
    if len(tree) == 0:
        print("Tree is empty")
    else:
        print("Binary Tree elements:", tree)

while True:
    print("\n1. Insert")
    print("2. Display")
    print("3. Exit")
    choice = int(input("Enter your choice: "))
    if choice == 1:
        value = int(input("Enter value: "))
        insert(value)
    elif choice == 2:
        display()
```

```
    elifchoice == 3:
        break
    else:
        print("Invalid choice")
```

Output:



Program:

```
# Binary Tree using Linked List
class Node:
    def __init__(self, data):
        self.data = data
        self.left = None
        self.right = None
root = Node(10)
root.left = Node(20)
root.right = Node(30)
root.left.left = Node(40)
root.left.right = Node(50)
def inorder(node):
    if node:
```

```
        inorder(node.left)
        print(node.data, end=" ")
        inorder(node.right)
def preorder(node):
    if node:
        print(node.data, end=" ")
        preorder(node.left)
        preorder(node.right)
def postorder(node):
    if node:
        postorder(node.left)
        postorder(node.right)
        print(node.data, end=" ")
print("Inorder Traversal:")
inorder(root)
print("\nPreorder Traversal:")
preorder(root)
print("\nPostorder Traversal:")
postorder(root)
```

Output:

```
AMD64)] on win32
Enter "help" below or click "Help" above for more informa
>>>
===== RESTART: C:\Users\Sruti\AppData\Local\Programs\Pytl
Binary Tree using Linked List:
20 10 30
```

Explanation:

- Binary Tree Using Array: Implements a binary tree using an array to insert, display, and search elements.
- Binary Tree Using Linked List: Implements a binary tree using nodes and performs inorder, preorder, and postorder traversals.

Error table:

Error	Retification
AttributeError: 'NoneType' object has no attribute 'left'	Check if node is not None: before accessing children.
NameError: name 'Node' is not defined	Define the Node class first.
Tree is full	Increase the array size or handle overflow.
None appears in output	Display only occupied positions if required.

LAB-9

1.AIM: Python program toimplementabinary tree with its operations and traversals.

~~Program:~~

```
# Binary Tree Operations and Traversals

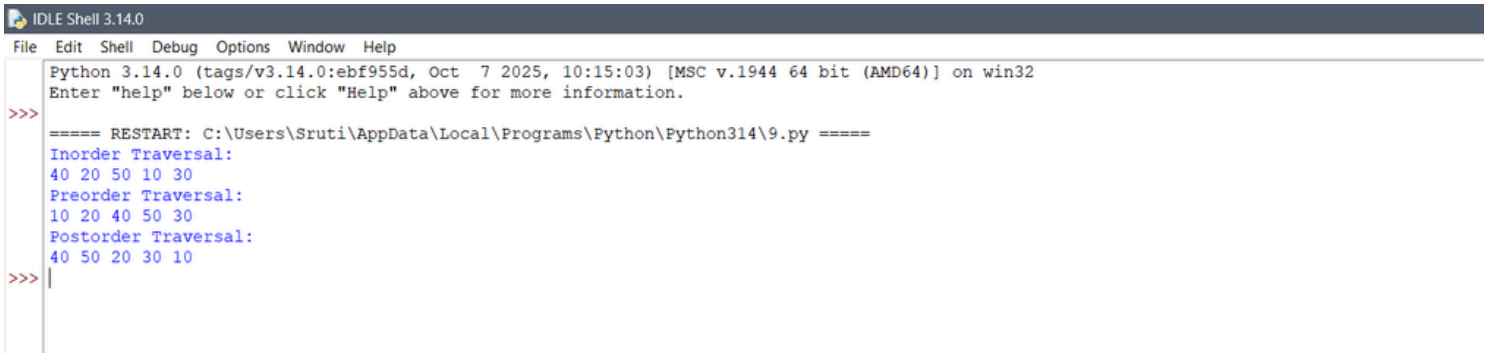
class Node:
    def __init__(self, data):
        self.data = data
        self.left = None
        self.right = None
def insert(root, data):
    if root is None:
        return Node(data)
    if data < root.data:
        root.left = insert(root.left, data)
    else:
        root.right = insert(root.right, data)
    return root
def inorder(root):
    if root:
        inorder(root.left)
        print(root.data, end=" ")
        inorder(root.right)
def preorder(root):
```

```
    if root:
print(root.data, end=" ")
        preorder(root.left)
        preorder(root.right)
def postorder(root):
    if root:
        postorder(root.left)
        postorder(root.right)
        print(root.data, end=" ")
def levelorder(root):
    if root is None:
        print("Tree is empty")
        return
    queue = [root]
    while queue:
        node = queue.pop(0)
        print(node.data, end=" ")
        if node.left:
            queue.append(node.left)
        if node.right:
            queue.append(node.right)
root = None
while True:
    print("\n\n1. Insert")
    print("2. Inorder")
```

```
print("3. Preorder")
print("4. Postorder")
print("5. Level Order")
print("6. Exit")
ch=int(input("Enter choice: "))
ifch== 1:
    value = int(input("Enter value: "))
    root = insert(root, value)
    print("Node inserted")
elifch == 2:
    print("Inorder:")
    inorder(root)
elifch == 3:
    print("Preorder:")
    preorder(root)
elifch == 4:
    print("Postorder:")
    postorder(root)
elifch == 5:
    print("Level Order:")
    levelorder(root)
elifch == 6:
```

```
print("Program ended")
        break
    else:
        print("Invalid choice")
```

Output:



Explanation:

Binary tree traversal is the process of visiting every node in a binary tree exactly once. It is split into two main categories: **Depth-First Search (DFS)** and **Breadth-First Search (BFS)**

Error table:

Error	Cause	Correction
Inorder is not sorted	The tree was not built using BST rules	Compare each inserted value with the current node.
RecursionError	Incorrect recursive calls or a cycle	Ensure recursive calls move to child nodes.
Infinite loop in the menu	Exit condition is missing or incorrect	Use break when the user selects Exit.
Wrong traversal result	Traversal calls are in the wrong order	Check the order of left, root, and right.